LITEUP · APPLICATION SECURITY

Can these vulnerabilities affect LiteUp?

Seven common web security issues, explained in plain language, with safe local checks and the expected LiteUp response.

Adewoye Mayowa
Cyber Security Student
SQI College of ICT · Osogbo Campus

LiteUp



A GUIDE FOR THE AUDIENCE

How to understand the checks

Each topic answers three questions: what the weakness means, how to check LiteUp safely, and what the response would look like if the protection were missing.

RED Vulnerable outcome

A hypothetical teaching example. It describes what could happen in an unsafe version; it is not evidence that LiteUp was exploited.

GREEN LiteUp result

Based on the project source review or documented local test. Each slide tells you which kind of evidence it is.

Browser checks

Use only localhost, a disposable database, dummy login details, and harmless files. Stop if the result differs from the expected safe response.

Source checks

File inclusion and command injection have no matching request-controlled feature in reviewed LiteUp code, so those are checked by reading/searching source.

01 File Upload | 02 File Inclusion | 03 CORS | 04 Brute Force | 05 Command Injection | 06 SQL Injection | 07 CSRF

Safe scope: These steps are for your own LiteUp installation. Do not use real customer accounts, real funds, or a public website for demonstrations.

SOURCE REVIEW · RUNTIME CHECK NOT RECORDED

File Upload Vulnerability

PROPER DEFINITION

A file upload vulnerability occurs when an application fails to safely validate, store, or serve user-supplied files, such as by trusting the filename, type, contents, or size.

IF UNPROTECTED

A harmful or oversized file could be stored or shown to users. Server execution depends on storage and server configuration.

LITEUP STATUS

Profile image code checks size, actual MIME type, image content, and generates a random filename. Funding proofs have separate type and size checks.

WHERE TO LOOK

`public/profile.php` and `public/fund_wallet.php`. Source review found validation; this presentation did not submit a new upload.

Safe local check · profile image

1. Sign in with a dummy account and open Profile.
2. Create a plain text file containing only `demo`; rename it `demo.jpg`.
3. Select and save it. Check the page message and whether the profile photo changed.

The file is harmless text, not executable code. Renaming tests whether the server checks the file's contents.

IF VULNERABLE · HYPOTHETICAL

The site may accept the text as an image and report success. Risky file types could create further server or user exposure.

LITEUP · EXPECTED

Reject with: “Please upload a JPG, PNG, GIF, or WebP image.” This is the source-based expected result, not a new runtime capture.

SAY: “I changed only the name of a harmless text file. LiteUp should inspect the contents and reject it.”

SOURCE REVIEW · NO REQUEST-CONTROLLED PATH FOUND

File Inclusion Vulnerability

PROPER DEFINITION

File inclusion is a vulnerability in which untrusted request data controls a file path passed to a server-side include operation, potentially loading an unintended local or remote file.

IF UNPROTECTED

Private content might be exposed, or unintended code could run, depending on what file is loaded and server settings.

LITEUP STATUS

Reviewed PHP pages use fixed application include paths. No request-controlled include path was found.

WHERE TO LOOK

Review `include` and `require` in `public/`, `admin/`, and `includes/`.

Safe check · source search

1. Open PowerShell in the LiteUp project folder.
2. Search PHP files for include paths built from request values:

```
Get-ChildItem .\public,.\admin,.\includes -Recurse -Filter *.php | Select-String -Pattern 'include\s+.*\$_|require\s+.*\$_'
```

Inspect any match; a search match alone does not prove a flaw. Do not try to read system files.

IF VULNERABLE · HYPOTHETICAL

If visitor input controls the loaded path, the page could include a file the developer did not intend.

LITEUP · REVIEW FINDING

No user-controlled include path was found. Because no such route exists, there is no safe file-inclusion payload to run on LiteUp.

SAY: “I checked who chooses the file path. LiteUp's code selects its own fixed files.”

LOCAL RESPONSE CHECK · NO ALLOW-ORIGIN FOUND

CORS Misconfiguration

PROPER DEFINITION

CORS is a browser-enforced policy that lets a server specify which origins may read its responses. A misconfiguration grants that access too broadly or to an untrusted origin.

IF UNPROTECTED

An overly broad policy can let an untrusted site's script read eligible LiteUp responses in a visitor's browser.

LITEUP STATUS

The reviewed application has no permissive CORS header; the local login response had no `Access-Control-Allow-Origin`.

WHERE TO LOOK

Check response headers from the local login page. This read-only request does not sign in or change data.

Safe check · PowerShell

1. Start Apache and open PowerShell.
2. Request the login page with a made-up Origin:

```
curl.exe -i -H "Origin: https://attacker.invalid"
"http://localhost/liteup_app/public/login.php"
```

3. Look for `Access-Control-Allow-Origin` in the headers.

IF VULNERABLE · HYPOTHETICAL

A response granting `*` or the untrusted origin could allow cross-site scripts to read eligible responses.

LITEUP · EXPECTED

No `Access-Control-Allow-Origin` header. No cross-site read permission is granted by the reviewed app.

> **SAY:** “I labeled a read-only request as coming from an untrusted site, then checked whether LiteUp granted it permission.”

LOCAL VERIFICATION · 5 ATTEMPTS, THEN BLOCKED

Brute-Force Attack

PROPER DEFINITION

A brute-force attack repeatedly submits password guesses against an account. Weak or missing rate limits make repeated attempts easier to continue.

IF UNPROTECTED

Unlimited guesses give an attacker more chances to take over an account.

LITEUP STATUS

Atomic rate limiting admits five attempts and blocks later attempts during a 15-minute window for the same source and identity.

WHERE TO LOOK

`public/login.php`, `includes/security.php`, and the recorded `test_rate_limiter.php` result.

Safe check · dummy identity only

1. Open the local login page. Use a test email or phone and an incorrect password.
2. Submit five times; note the response.
3. Submit once more and note whether it is rate-limited. Stop after attempt six.

Use no real account. Wait 15 minutes or use a new dummy identity before repeating.

IF VULNERABLE · HYPOTHETICAL

Every guess continues to reach password checking. Persistent guessing increases account-takeover risk.

LITEUP · RECORDED RESULT

Attempts 1–5 are admitted; attempt 6 is limited with “Invalid login details. Please try again later.”

SAY: “LiteUp allows five tries, then slows repeated guessing for that test identity.”

SOURCE REVIEW · NO SHELL SINK FOUND

Command Injection

PROPER DEFINITION

Command injection occurs when untrusted input is incorporated into an operating-system command and changes the command or arguments executed by the server.

IF UNPROTECTED

The server could run an unintended command, expose data, or alter files within the web-server account's access.

LITEUP STATUS

No PHP shell-execution API was found in the reviewed application files.

WHERE TO LOOK

Search PHP source, then trace any match to see whether request input reaches it. A match alone is not proof.

Safe check · source search only

1. Open PowerShell in the project folder.
2. Search the application PHP files for common shell functions:

```
Get-ChildItem .\public,.\admin,.\includes -Recurse -Filter *.php | Select-String -Pattern 'shell_exec|exec\s*\(|system\s*\(|passthru|proc_open|popen'
```

Do not submit command strings to the website. This app has no command feature to test.

IF VULNERABLE · HYPOTHETICAL

If untrusted text reaches a shell unsafely, it may cause an unauthorized server action.

LITEUP · REVIEW FINDING

No shell command sink was found. There is no safe payload to run against the current app.

SAY: “I searched for PHP functions that start system commands and checked whether user input can reach them. None were found.”

LOCAL TEST · INVALID LOGIN, NO SESSION

SQL Injection

PROPER DEFINITION

SQL injection is a flaw where untrusted input is combined with a SQL query as executable syntax instead of being handled only as data.

IF UNPROTECTED

A login could be bypassed, or database records could be read or changed.

LITEUP STATUS

The login uses prepared statements. The recorded local test returned an ordinary invalid-login message and created no session.

WHERE TO LOOK

`public/login.php` binds login identity values separately from SQL.

Safe check · disposable local database

1. Use only a dummy account on a disposable local database.
2. In the login identity field enter `' OR 1=1 --` ; use any wrong password.
3. Submit once. Check the message and verify no login session was created.

Stop if a sign-in succeeds or a database error appears. This uses one rate-limit attempt.

IF VULNERABLE · SIMULATED

Raw input may alter SQL logic. A sign-in bypass or database error could reveal an unsafe query.

LITEUP · CAPTURED LOCAL TEST

“Invalid login details.” No session created. Prepared parameters keep the input as data.



SAY: “LiteUp treated the unusual text as an invalid identity, not as database instructions.”

LOCAL TEST · GET 405, TOKENLESS POST 403

Cross-Site Request Forgery (CSRF)

PROPER DEFINITION

CSRF is an attack that makes an authenticated user's browser send an unintended state-changing request, relying on the browser's existing session credentials.

IF UNPROTECTED

A website could cause an unwanted state-changing action using the visitor's active session.

LITEUP STATUS

State-changing forms use session tokens. Logout requires POST and checks the token.

WHERE TO LOOK

`public/logout.php`, `includes/security.php`, and the captured local response.

Safe check · dummy account only

1. Sign in locally with the dummy account and open `/public/logout.php` directly in the same browser; confirm the session remains active.
2. In PowerShell, send a tokenless POST:

```
curl.exe -i -X POST "http://localhost/liteup_app/public/logout.php"
```

3. Read the HTTP status and message. The curl request uses its own session.

IF VULNERABLE · HYPOTHETICAL

A request without the user's intent or a valid token may perform the action. In the old logout pattern, a GET could destroy the session.

LITEUP · CAPTURED LOCAL TEST

GET returns **405**; POST without a valid token returns **403**. The normal logout form carries the token.

SAY: “LiteUp checks both that logout uses POST and that it includes the user's session token.”

Captured evidence: docs/evidence-csrf-logout.png · public/logout.php

SUMMARY · WHAT LITEUP SHOWS

Which issues were found, tested, or reviewed?

File uploads · source review

Server-side checks are present. A harmless renamed-text-file upload is the suggested runtime check; not recorded here.

File inclusion · source review

No request-controlled include path found. Fixed paths are used.

CORS · local response check

No permissive allow-origin header found on the login response.

Brute forcing · local verification

Five attempts allowed, then later attempts are limited within 15 minutes.

Command injection · source review

No shell-execution sink found in the reviewed app files.

SQL injection · local test

Login input was rejected normally; no session was created.

CSRF · local test

Logout GET was rejected with 405; tokenless POST was rejected with 403.

Evidence rule

Hypothetical vulnerable outcomes are teaching illustrations. Source review is not the same as live proof.

Conclusion: LiteUp has useful protections for the reviewed paths. Keep testing only on localhost with dummy accounts, and review any new endpoints before deployment.

REFERENCES

Project evidence and security guidance

Project-specific claims refer to the LiteUp source or local verification records. External guidance supports the plain-language definitions and prevention concepts.

LiteUp source and test records
SECURITY-STATUS.md · TESTING-GUIDE.md · public/login.php · public/profile.php · public/logout.php · includes/security.php · test_rate_limiter.php

OWASP · File Upload Cheat Sheet
File upload security guidance

OWASP · SQL Injection Prevention
Prepared statement guidance

OWASP · CSRF Prevention
CSRF token guidance

OWASP · OS Command Injection Defense
Command injection defense

OWASP · Authentication
Authentication and login guidance

OWASP · HTTP Headers
CORS and response header guidance

PHP Manual · include
PHP include behavior